UNIVERSIDADE FEDERAL DE SANTA MARIA

Programa de Pós-Graduação / Iniciação Científica

UFSM-Reviewer

Plataforma Multi-Agente Local para Apoio à Produção e Revisão Científica Universitária

Pré-Projeto de Pesquisa e Desenvolvimento

Coordenadora: Andrea Schwertner Charão

Equipe: Grupo Verdi

Programa: Interdisciplinar

Área: Inteligência Artificial / Informática na Educação

Santa Maria, Rio Grande do Sul
2026

Sumário

Resumo **2**

1 Introdução **3**
1.1 Contextualização . . . 3
1.2 Objetivos . . . 3
1.3 Justificativa . . . 4
1.4 Possíveis Evoluções . . . 4

2 Revisão Bibliográfica **6**
2.1 Revisão Científica Automatizada . . . 6
2.2 Frameworks Multi-Agente com Recuperação de Contexto . . . 6
2.3 Modelos de Linguagem Open-Source Locais . . . 6
2.4 Busca e Recuperação de Literatura Científica . . . 6
2.5 Frameworks de Orquestração Multi-Agente . . . 7
2.6 Avaliação de Revisões Científicas Automatizadas . . . 7

3 Metodologia **8**
3.1 Arquitetura do Sistema . . . 8
3.2 Stack Tecnológica . . . 8
3.3 Necessidade de Hardware . . . 9
3.4 Necessidade de Pessoal . . . 10
3.5 Roteiro de Implementação . . . 10
3.6 Roteiro de Testes . . . 12
3.7 Definição de Atividades e Tarefas por Responsável . . . 13

4 Cronograma Físico **14**

5 Resultados Esperados **15**

6 Conclusões **15**

Resumo

A produção científica de qualidade no ambiente universitário enfrenta desafios estruturais relacionados à falta de mecanismos de *feedback* iterativo antes da submissão de trabalhos a periódicos e conferências. Este projeto propõe o desenvolvimento do **UFSM-Reviewer**, uma plataforma multi-agente baseada em modelos de linguagem de grande escala (LLMs) executados localmente via Ollama, integrada à Semantic Scholar API, capaz de simular o processo de revisão por pares sobre artigos produzidos por alunos de graduação e pós-graduação. O sistema não apenas aponta pontos fortes e fracos dos manuscritos, mas também **explica pedagogicamente cada crítica gerada**, conectando-a à literatura científica pertinente e auxiliando o aluno na interpretação e no aperfeiçoamento do trabalho. Por operar inteiramente dentro da infraestrutura da instituição, o projeto garante total privacidade dos manuscritos ainda não publicados. A plataforma será validada por meio de simulações de revisão por pares com especialistas humanos, permitindo avaliar a qualidade e a utilidade pedagógica das revisões geradas.

Palavras-chave: revisão científica automatizada; sistemas multi-agente; modelos de linguagem local; Ollama; formação de pesquisadores; revisão por pares.

1 Introdução

1.1 Contextualização

O processo de produção científica de qualidade exige ciclos contínuos de avaliação crítica e refinamento. No entanto, a relação orientador-orientando, especialmente nos níveis de graduação e início da pós-graduação, frequentemente não comporta a revisão detalhada e iterativa de múltiplos rascunhos de um mesmo trabalho. O resultado prático é que muitos artigos chegam à submissão com deficiências metodológicas evitáveis: ausência de comparações com o estado da arte, alegações de novidade não fundamentadas na literatura recente, inconsistências entre objetivos e resultados, e estrutura argumentativa frágil.

Paralelamente, a comunidade científica global vivencia uma crise de escalabilidade no processo de revisão por pares: conferências de alto impacto como NeurIPS e ICLR recebem dezenas de milhares de submissões anuais, gerando fadiga nos revisores e alta variância na qualidade das avaliações recebidas [6]. Esse cenário motivou uma crescente linha de pesquisa em revisão científica automatizada assistida por inteligência artificial, culminando em *frameworks* como o ScholarPeer [4], que demonstrou ser possível superar a qualidade média de revisores humanos em dimensões como avaliação de novidade e identificação de *baselines* ausentes.

O presente projeto adapta e estende esses avanços para o contexto pedagógico universitário, priorizando a explicabilidade das críticas geradas, a execução local dos modelos e a validação rigorosa por simulação de revisão por pares.

1.2 Objetivos

Objetivo Geral

Desenvolver, implantar e validar uma plataforma multi-agente de revisão científica automatizada, executada localmente, capaz de fornecer *feedback* estruturado, contextualizado e pedagogicamente explicado sobre manuscritos acadêmicos produzidos por alunos universitários.

Objetivos Específicos

1. Implementar uma arquitetura multi-agente baseada em LLMs *open-source* via Ollama, integrando busca de literatura em tempo real pela Semantic Scholar API;

2. Desenvolver uma camada explicativa que transforme cada crítica gerada em material de aprendizagem, referenciando a literatura científica que fundamenta o apontamento;

3. Implementar um módulo de simulação de revisão por pares que permita comparar as revisões geradas pela plataforma com avaliações de especialistas humanos;
4. Validar a qualidade pedagógica da plataforma por meio de experimentos controlados com alunos de graduação e pós-graduação;
5. Publicar os resultados da validação e disponibilizar o código-fonte como contribuição à comunidade acadêmica.

1.3 Justificativa

A justificativa para o desenvolvimento do UFSM-Reviewer articula-se em três dimensões:

Dimensão Pedagógica: Ferramentas que simulam o processo de revisão por pares ensinam, na prática, o que caracteriza um bom artigo científico. Ao receber críticas fundamentadas e explicadas, o aluno não apenas melhora o trabalho em questão, mas internaliza os critérios de qualidade científica de forma ativa.

Dimensão Institucional: A plataforma reduz a carga de revisões repetitivas sobre rascunhos iniciais pelos orientadores, liberando tempo para discussões de alto nível sobre contribuição científica e direcionamento de pesquisa.

Dimensão de Soberania e Privacidade: A execução inteiramente local dos modelos via Ollama elimina o envio de manuscritos não publicados a servidores externos, preservando a confidencialidade dos trabalhos em desenvolvimento — aspecto crítico em projetos com potencial de propriedade intelectual ou em fase pré-publicação.

1.4 Possíveis Evoluções

O projeto foi concebido com arquitetura modular e expansível. Entre as evoluções previstas para fases futuras destacam-se:

- **Suporte multilíngue aprimorado:** *fine-tuning* dos modelos para revisão de artigos em português científico, com terminologia específica por área do conhecimento;
- **Módulo de debate entre agentes:** implementação de revisões por múltiplos agentes com perfis distintos (cético, construtivista, especialista em metodologia), simulando um comitê de revisão;
- **Integração com plataformas de submissão institucional:** conexão com sistemas de TCC e dissertações já utilizados pela universidade;

- **Módulo de análise de evolução:** rastreamento da melhoria da qualidade dos manuscritos ao longo das versões, gerando dados para pesquisa sobre aprendizagem científica;
- **Base de dados de revisões anonimizadas:** geração de um corpus nacional de revisões em português para pesquisa em PLN aplicada à ciência.

2 Revisão Bibliográfica

2.1 Revisão Científica Automatizada

A automação da revisão científica evoluiu de abordagens de classificação de texto para geração estruturada de *feedback*. Trabalhos seminais como [7] utilizaram modelos pré-treinados para predição de decisões editoriais. Com o aumento da escala dos LLMs, o foco deslocou-se para o *fine-tuning* supervisionado sobre *datasets* de revisões reais: CycleReviewer [11] introduziu restrições negativas para melhorar a qualidade das críticas; DeepReviewer [13] aplicou *fine-tuning* com *chain-of-thought* para aprofundar o raciocínio analítico. Contudo, todos esses sistemas compartilham uma limitação fundamental: avaliam artigos em isolamento, sem acesso ao contexto externo que um especialista humano possui — o chamado "problema do vácuo paramétrico".

2.2 Frameworks Multi-Agente com Recuperação de Contexto

Para superar a limitação do vácuo paramétrico, *frameworks* agênticos integraram recuperação aumentada de geração (RAG). AgentReview [5] modelou o processo de revisão como discussão multi-estágio entre agentes. ReviewRL [12] e REMOR [10] integraram aprendizado por reforço para alinhar revisões às preferências humanas. O ScholarPeer [4] representa o estado da arte, combinando busca *web* em escala com agentes especializados em contextualização histórica e auditoria adversarial de *baselines*, atingindo H-Max Score de 6,14 (onde 5,0 equivale à qualidade do melhor revisor humano) em avaliação sobre 1.286 artigos do ICLR.

2.3 Modelos de Linguagem Open-Source Locais

A viabilidade de execução local de LLMs competitivos foi radicalmente transformada pela disponibilização de modelos quantizados de alta qualidade. O Ollama [8] consolidou-se como plataforma padrão para execução local de LLMs, oferecendo API REST compatível com o formato OpenAI e suporte a GPU via CUDA. Modelos como Llama 3.3 70B (Meta, 2024), Qwen3 14B (Alibaba, 2025) e Gemma3 12B (Google, 2025) demonstraram desempenho competitivo com modelos proprietários em tarefas de raciocínio e análise de textos científicos, especialmente em configurações quantizadas de 4 bits (q4) que reduzem requisitos de VRAM sem degradação significativa de qualidade.

2.4 Busca e Recuperação de Literatura Científica

A Semantic Scholar API [9] (Allen Institute for AI) disponibiliza acesso gratuito a mais de 200 milhões de artigos científicos, com suporte a busca semântica, recuperação de

metadados completos, grafo de citações e identificação de artigos relacionados. Diferentemente de APIs pagas como Elsevier ou Springer, não exige chave de autenticação para uso básico (até 100 requisições por 5 minutos), tornando-a ideal para projetos institucionais com restrições orçamentárias.

2.5 Frameworks de Orquestração Multi-Agente

CrewAI (2024–presente) e LangGraph (LangChain, 2024) são os principais *frameworks* Python para orquestração de sistemas multi-agente. CrewAI adota um paradigma orientado a papéis (*roles*), onde cada agente possui objetivo, ferramentas e *backstory* definidos, comunicando-se por meio de tarefas sequenciais ou paralelas. Ambos oferecem integração nativa com Ollama por meio da compatibilidade com a interface OpenAI, eliminando dependências de APIs proprietárias.

2.6 Avaliação de Revisões Científicas Automatizadas

A avaliação de sistemas de revisão automatizada apresenta desafios metodológicos específicos. Goyal et al. [4] introduziram o **H-Max Score**, que calibra a qualidade da revisão IA em relação ao conjunto coletivo de revisões humanas para um mesmo artigo, e o **Review Diversity Score**, que mede a variância semântica entre revisões geradas para o mesmo trabalho — métrica importante para detectar homogeneização de opiniões. ReviewEval [3] propõe um *framework* abrangente para avaliar acionabilidade e profundidade das revisões geradas.

3 Metodologia

3.1 Arquitetura do Sistema

O UFSM-Reviewer é organizado em cinco agentes especializados, orquestrados via CrewAI, todos executados sobre modelos locais via Ollama. A Figura 1 apresenta o diagrama completo do sistema.

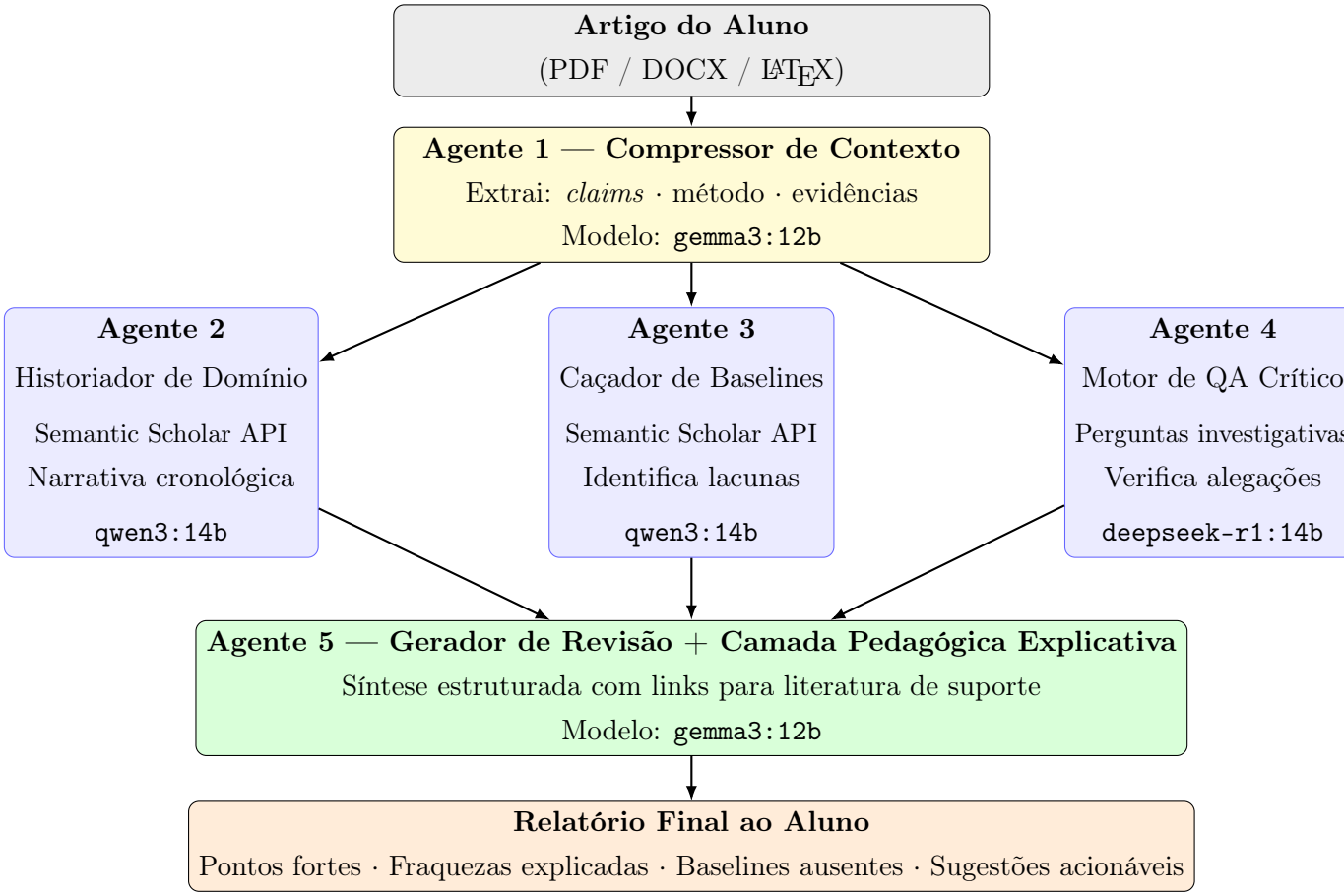

Figura 1: Arquitetura do UFSM-Reviewer: pipeline multi-agente com camada pedagógica explicativa.

A **Camada Pedagógica Explicativa** diferencia o UFSM-Reviewer do ScholarPeer: o Agente 5 transforma cada crítica técnica em três componentes para o aluno: (a) descrição do problema identificado; (b) explicação de *por que* isso é um problema científico, com referência ao artigo ou padrão metodológico que o fundamenta; e (c) sugestão concreta e acionável de como corrigi-lo.

3.2 Stack Tecnológica

Todo o projeto é construído sobre tecnologias gratuitas e *open-source*, conforme detalhado na Tabela 1.

Tabela 1: Stack tecnológica do UFSM-Reviewer.

Componente	**Tecnologia**	**Licença**
Execução de LLMs	Ollama	MIT — gratuito
Orquestração de agentes	CrewAI (Python)	MIT — gratuito
Busca de literatura	Semantic Scholar API	Gratuita
Embeddings vetoriais	nomic-embed-text via Ollama	Apache 2.0
Banco vetorial	PostgreSQL + pgvector	MIT — gratuito
Interface web	Streamlit	Apache 2.0
Backend API	FastAPI (Python)	MIT — gratuito
Containerização	Docker + Docker Compose	Apache 2.0
Parsing de documentos	PyMuPDF + Pandoc	Open-source
Controle de versão	Git + GitHub/GitLab	Gratuito

Modelos LLM via Ollama

A Tabela 2 apresenta os modelos selecionados para cada agente, com os respectivos requisitos de VRAM.

Tabela 2: Modelos LLM via Ollama por agente.

Agente	**Modelo**	**Tamanho**	**VRAM mínima**
Compressor e Gerador (Ag. 1 e 5)	`gemma3:12b`	∼8 GB	8 GB
Historiador e Caçador (Ag. 2 e 3)	`qwen3:14b`	∼9 GB	10 GB
Motor de QA (Ag. 4)	`deepseek-r1:14b`	∼9 GB	10 GB
Embeddings	`nomic-embed-text`	∼270 MB	1 GB

3.3 Necessidade de Hardware

Configuração Mínima

- GPU NVIDIA com 16 GB VRAM (ex.: RTX 3080, RTX 4080 ou equivalente em *workstation*)
- 32 GB RAM
- 500 GB SSD NVMe
- CPU: 8 *cores* ou mais

Configuração Recomendada

- GPU NVIDIA com 24 GB VRAM (ex.: RTX 3090, RTX 4090 ou A5000)
- 64 GB RAM
- 1 TB SSD NVMe
- CPU: 16 *cores* ou mais
- Rede: 1 Gbps para acesso interno

Alternativa sem GPU

Para ambientes sem GPU dedicada, é viável operar com modelos menores (`gemma3:4b`, `phi4-mini`) em CPU, com *throughput* reduzido (∼1 artigo/hora), adequado para fase de testes.

3.4 Necessidade de Pessoal

Tabela 3: Equipe necessária para o projeto.

Função	Perfil	Dedicação
Coordenador do Projeto	Professor com experiência em IA/PLN	4 h/semana
Desenvolvedor Principal	Aluno de mestrado ou doutorando em computação	20 h/semana
Desenvolvedor Auxiliar	Aluno de IC em ciência da computação	12 h/semana
Especialista de Domínio (×2)	Professores das áreas participantes	2 h/semana
Avaliador Externo (×5)	Pesquisadores revisores para validação (Fase 3)	Pontual

3.5 Roteiro de Implementação

Fase 1 — Infraestrutura e Prova de Conceito (Meses 1–2)

At. 1.1 Configuração do ambiente Docker Compose com Ollama, PostgreSQL/pgvector e Streamlit

At. 1.2 Pull e teste dos modelos: `gemma3:12b`, `qwen3:14b`, `deepseek-r1:14b`, `nomic-embed-text`

At. 1.3 Implementação do Agente 1 (Compressor de Contexto) com *parser* de PDF/LaTeX

At. 1.4 Implementação do Agente 4 (Motor de QA) com geração de perguntas críticas

At. 1.5 Interface Streamlit básica para *upload* e visualização de resultados

At. 1.6 Teste com 10 artigos coletados de edições anteriores de eventos da universidade

Fase 2 — Integração de Literatura e Agentes Especializados (Meses 3–4)

At. 2.1 Integração com a Semantic Scholar API para busca dinâmica de literatura

At. 2.2 Implementação do Agente 2 (Historiador de Domínio) com narrativa cronológica

At. 2.3 Implementação do Agente 3 (Caçador de Baselines) com auditoria adversarial

At. 2.4 Construção do banco vetorial (pgvector) com *embeddings* da literatura recuperada

At. 2.5 Implementação da Camada Pedagógica Explicativa no Agente 5

At. 2.6 Integração completa do *pipeline* e testes de ponta a ponta

Fase 3 — Validação por Simulação de Revisão por Pares (Meses 5–6)

At. 3.1 Coleta de 50 artigos com revisões humanas reais (eventos internos, anais, *preprints*)

At. 3.2 Geração de revisões automatizadas pelo UFSM-Reviewer para os 50 artigos

At. 3.3 Avaliação cega por revisores especialistas: comparação revisão UFSM-Reviewer × revisão humana usando protocolo H-Max Score adaptado

At. 3.4 Avaliação pedagógica com alunos: experimento controlado, grupo com ferramenta × grupo controle, medindo melhoria de qualidade na segunda versão do artigo

At. 3.5 Análise estatística dos resultados e iteração final sobre o sistema

Fase 4 — Documentação, Publicação e Implantação (Meses 7–8)

At. 4.1 Refinamento da interface com base no *feedback* dos experimentos

At. 4.2 Documentação técnica completa e guia de implantação

At. 4.3 Disponibilização do código-fonte em repositório público

At. 4.4 Escrita e submissão do artigo científico com os resultados da validação

At. 4.5 Implantação definitiva na infraestrutura institucional

3.6 Roteiro de Testes

Testes de Unidade (Fases 1–2)

Cada agente é testado individualmente com entradas controladas. Métricas avaliadas: latência de resposta, coerência da saída e taxa de alucinações factuais verificáveis.

Testes de Integração (Fase 2)

O *pipeline* completo é executado sobre artigos com falhas conhecidas e injetadas (baseline ausente, novidade não fundamentada, metodologia inconsistente). Verifica-se se cada falha injetada é detectada pelo agente correspondente.

Testes de Validação por Pares (Fase 3)

- **Protocolo A — Qualidade da Revisão:** 5 especialistas avaliam às cegas revisões do UFSM-Reviewer e revisões humanas reais, usando escala H-Max adaptada (1–10);
- **Protocolo B — Eficácia Pedagógica:** 30 alunos divididos em grupo experimental (com UFSM-Reviewer) e grupo controle; ambos reescrevem seus artigos; avaliadores externos comparam as segundas versões sem conhecer o grupo de origem;
- **Protocolo C — Satisfação e Usabilidade:** questionário SUS (*System Usability Scale*) aplicado aos alunos participantes.

3.7 Definição de Atividades e Tarefas por Responsável

Tabela 4: Responsabilidades por atividade.

Atividade	**Responsável Principal**	**Apoio**
Configuração de infraestrutura Docker	Desenvolv. Principal	Desenvolv. Auxiliar
Implementação dos agentes CrewAI	Desenvolv. Principal	Coordenador (revisão)
Integração Semantic Scholar API	Desenvolv. Auxiliar	Desenvolv. Principal
Camada pedagógica explicativa	Coordenador + Desenvolv. Principal	Especialistas
Interface Streamlit	Desenvolv. Auxiliar	—
Coleta de artigos para validação	Especialistas de Domínio	Coordenador
Protocolo de avaliação por pares	Coordenador	Avaliadores Externos
Análise estatística	Coordenador	Desenvolv. Principal
Escrita do artigo científico	Coordenador + Desenvolv. Principal	Todos

4 Cronograma Físico

Tabela 5: Cronograma físico de atividades (8 meses).

Atividade	**M1**	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**
At. 1.1 Ambiente Docker + Ollama	•							
At. 1.2 Pull e teste dos modelos	•							
At. 1.3 Agente 1 — Compressor	•	•						
At. 1.4 Agente 4 — Motor de QA		•						
At. 1.5 Interface Streamlit básica		•						
At. 1.6 Testes com artigos históricos		•						
At. 2.1 Integração Semantic Scholar			•					
At. 2.2 Agente 2 — Historiador			•					
At. 2.3 Agente 3 — Caçador Baselines			•	•				
At. 2.4 Banco vetorial pgvector				•				
At. 2.5 Camada Pedagógica Explicativa			•	•				
At. 2.6 Integração e testes completos				•				
At. 3.1 Coleta de artigos validação					•			
At. 3.2 Geração revisões automatizadas					•			
At. 3.3 Avaliação por especialistas					•	•		
At. 3.4 Experimento pedagógico alunos						•		
At. 3.5 Análise estatística						•		
At. 4.1 Refinamento da interface							•	
At. 4.2 Documentação técnica							•	
At. 4.3 Disponibilização open-source								•
At. 4.4 Escrita e submissão do artigo							•	•
At. 4.5 Implantação institucional								•

5 Resultados Esperados

Ao término do projeto, espera-se que o UFSM-Reviewer entregue os seguintes resultados concretos:

1. **Plataforma funcional e implantada:** sistema operacional na infraestrutura da universidade, acessível por alunos e professores via interface *web*, capaz de processar artigos em formato PDF, DOCX e LaTeX e gerar revisões estruturadas em português;
2. **Qualidade de revisão validada:** nas avaliações por especialistas humanos, espera-se que o UFSM-Reviewer atinja H-Max Score médio superior a 4,5, com base nos *benchmarks* reportados por [4] para sistemas similares;
3. **Eficácia pedagógica demonstrada:** artigos do grupo experimental apresentam, em suas segundas versões, melhoria estatisticamente significativa em qualidade metodológica e fundamentação bibliográfica em relação ao grupo controle;
4. **Privacidade garantida:** 100% do processamento realizado localmente, sem envio de dados a APIs externas, auditável por qualquer membro da instituição;
5. **Artigo científico publicado:** submissão de artigo relatando a arquitetura, a implementação e os resultados da validação a periódico ou conferência nacional/internacional na área de Informática na Educação ou Inteligência Artificial;
6. **Código *open-source* disponível:** repositório público com documentação, guia de implantação e *scripts* de configuração, permitindo adoção por outras instituições;
7. **Corpus de revisões anonimizadas:** banco de dados de revisões geradas e avaliadas, disponível para pesquisas futuras em PLN aplicado à escrita científica em português.

6 Conclusões

O UFSM-Reviewer representa uma resposta concreta e viável a dois desafios simultaneamente presentes no ambiente universitário brasileiro: a necessidade de mecanismos de *feedback* de qualidade na formação de pesquisadores e a crescente disponibilidade de tecnologias de IA que, até recentemente, estavam restritas a grandes corporações tecnológicas. A opção por modelos *open-source* executados localmente via Ollama não é apenas uma escolha de custo — é uma decisão estratégica que garante soberania sobre os dados, independência de fornecedores externos e sustentabilidade de longo prazo da plataforma na instituição.

A distinção fundamental em relação ao estado da arte reside na **camada pedagógica explicativa**: enquanto sistemas como o ScholarPeer [4] foram projetados para especialistas que já conhecem o contexto científico e precisam de uma segunda opinião eficiente, o UFSM-Reviewer foi concebido para *formar pesquisadores*, explicando *por que* cada crítica é relevante e *como* o aluno pode superá-la. A validação por simulação de revisão por pares com especialistas humanos garante que essa proposta seja avaliada com o mesmo rigor metodológico que o próprio sistema exige dos artigos que revisa.

Referências

[1] CHITALE, M. P. et al. **AutoRev: Automatic peer review system for academic research papers**. arXiv:2505.14376, 2025.

[2] DHULIAWALA, S. et al. Chain-of-verification reduces hallucination in large language models. In: **Findings of the ACL 2024**, p. 3563–3578, 2024.

[3] GARG, M. K. et al. **ReviewEval: An evaluation framework for AI-generated reviews**. arXiv:2502.11736, 2025.

[4] GOYAL, P. et al. **ScholarPeer: A Context-Aware Multi-Agent Framework for Automated Peer Review**. arXiv:2601.22638v1, 2026.

[5] JIN, Z. et al. **AgentReview: Exploring peer review dynamics with LLM agents**. arXiv:2406.12708, 2024.

[6] LIANG, W. et al. Can large language models provide useful feedback on research papers? A large-scale empirical analysis. **NEJM AI**, 2024.

[7] LIU, N.; SHAH, N. **ReviewerGPT? An exploratory study on using large language models for paper reviewing**. arXiv:2306.00622, 2023.

[8] OLLAMA. **Ollama: Get up and running with large language models**. Disponível em: https://ollama.com. Acesso em: abr. 2026.

[9] SEMANTIC SCHOLAR. **Semantic Scholar Academic Graph API**. Allen Institute for AI. Disponível em: https://api.semanticscholar.org. Acesso em: abr. 2026.

[10] TAECHOYOTIN, P.; ACUNA, D. E. **REMOR: Reinforcement learning from multi-aspect online reviews for automated paper reviewing**. arXiv, 2025.

[11] WENG, Y. et al. **CycleReviewer: Exploring the potential of large language models in automated peer review with a progressive training approach**. arXiv:2412.18037, 2024.

[12] ZENG, A. et al. **ReviewRL: Optimizing LLM reviewers via reinforcement learning**. arXiv, 2025.

[13] ZHU, Z. et al. **DeepReviewer: Collaborative deep thinking for peer review**. arXiv, 2025.